



OpenTelemetry: The Future of Cloud Observability

Comprehensive Team Presentation & Implementation Guide



1. Executive Summary: The "Multi-Cloud" Challenge

The Problem: Fragmentation

In our current landscape (AWS, Azure, On-Prem/Local), we suffer from **Vendor Lock-in** and **Data Silos**:

- **AWS**: CloudWatch Agent (Proprietary)
- **Azure**: Azure Monitor Agent (Proprietary)
- **Local**: Custom Logging (Manual)

The Solution: OpenTelemetry (OTel)

OTel is a **CNCF Standard** that provides a single, unified way to collect metrics, logs, and traces.

1. **Instrument Once**: Use the OTel SDK in your code once.
2. **Collect Anywhere**: The OTel Collector acts as a central router.
3. **Route Everywhere**: Send data to Prometheus, Datadog, Splunk, or Azure Monitor without changing a single line of application code.

2. Project Architecture Breakdown

This demo simulates a real cloud-native environment using Docker.

A. The Application Layer (Python + Ubuntu)

We simulate a real Ubuntu server running a Python script that monitors system health.

Dockerfile.ubuntu

Code Snippet	Explanation
<code>FROM ubuntu:22.04</code>	We use a real Linux base (Ubuntu) to mirror a production VM environment.
<code>ENV PYTHONUNBUFFERED=1</code>	CRITICAL: Default Python buffers logs. This makes logs appear instantly in Docker logs for monitoring.
<code>RUN apt-get update && apt-get install -y python3 python3-pip</code>	Standard OS hardening; installing <code>pip</code> to manage our OTEL libraries.
<code>RUN pip3 install -r requirements.txt</code>	Installs the OTEL SDK and <code>psutil</code> (which gives us real hardware access).

metric_generator.py (The Instrumentation Heart)

This is where the magic happens. We instrument the code using the OTEL SDK.

1. Resource Identification

```
resource = Resource.create({
    "service.name": "ubuntu-demo-app",
    "host.name": "ubuntu-vm-1"
})
```

Defines the "identity" of the metrics. When these reach the cloud, they will be tagged with `host.name: ubuntu-vm-1` so you know exactly which server is failing.

2. Setup the Exporter

```
endpoint = os.environ.get("OTEL_EXPORTER_OTLP_ENDPOINT", "http://otel-collector:4317")
exporter = OTLPMetricExporter(endpoint=endpoint, insecure=True)
```

Configures the "Push" mechanism. We use **gRPC** (Port 4317). We use `insecure=True` for internal Docker networking.

3. Fetching Real Data

```
def get_cpu_info(options):
    cpu_percent = psutil.cpu_percent(interval=None)
    yield metrics.Observation(cpu_percent)

def get_mem_info(options):
    memory_info = psutil.virtual_memory()
    mem_mb = memory_info.used / (1024 * 1024)
    yield metrics.Observation(mem_mb)
```

These functions leverage `psutil` to fetch **real** real-time CPU % and Memory Usage (MB).

4. Registering Gauges

```
cpu_gauge = meter.create_observable_gauge(
    name="cpu_usage_percentage",
    callbacks=[get_cpu_info],
)
```

We create "Observable Gauges." Unlike counters, gauges represent a value that can go up or down.

B. The Routing Layer (OTel Collector)

The Collector is the world's most flexible observability tool.

otel-collector-config.yaml

1. Receivers (The Front Door)

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: "0.0.0.0:4317"
```

Defines where data enters. We listen for gRPC on port 4317.

2. Exporters (The Back Door)

```
exporters:
  prometheus:
    endpoint: "0.0.0.0:8889"
```

Translates OTel data into Prometheus format and hosts it on port 8889.

3. Service Pipeline (The Glue)

```
service:
  pipelines:
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheus, debug]
```

Connects everything securely.

C. The Visual Stack (Prometheus & Grafana)

prometheus.yml

```
scrape_configs:  
  - job_name: 'otel-collector'  
    static_configs:  
      - targets: ['otel-collector:8889']
```

Configured to "scrape" the Collector every 5 seconds.

datasource.yml

```
datasources:  
  - name: Prometheus  
    url: http://prometheus:9090
```

Automates the connection between Grafana and Prometheus.

3. End-to-End Verification Guide

Step 1: Bootstrap the Environment

```
docker-compose up --build -d
```

Step 2: Verify Application Connectivity

Check if the Ubuntu app is talking to the Collector:

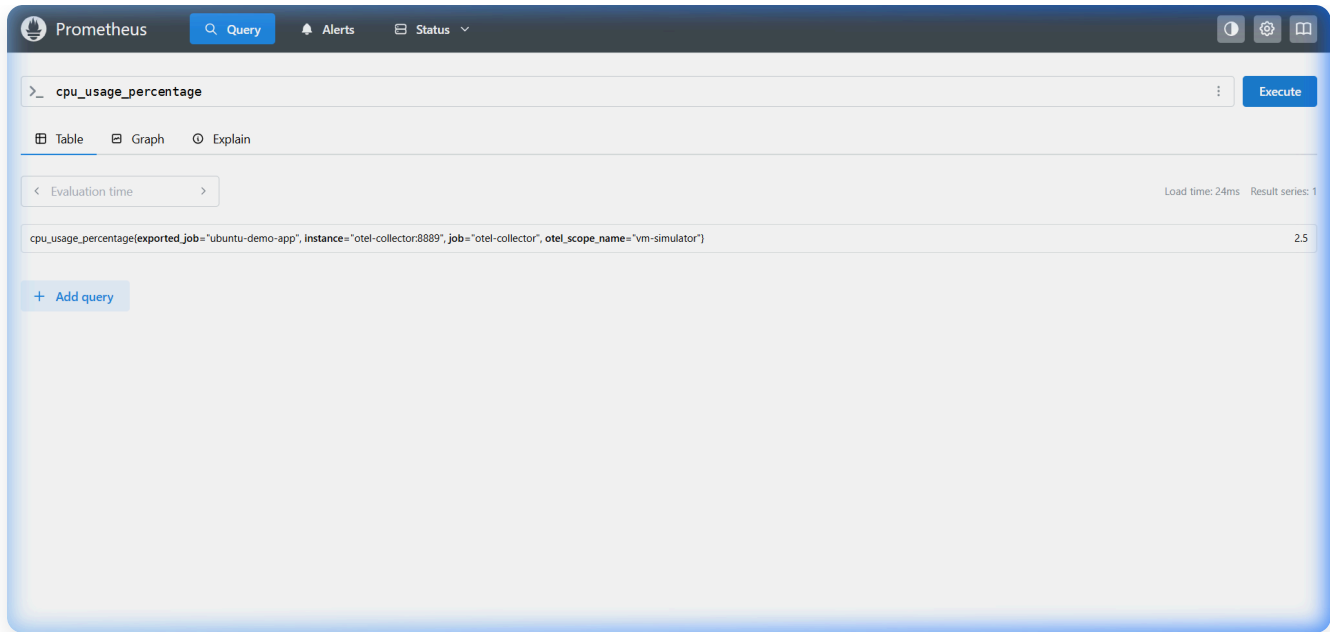
```
docker logs -f otel-demo-ubuntu-app-1
```

Expected Output: *Sending metrics to OTEL Collector via gRPC at: http://otel-collector:4317*

Step 3: Verify Metric Retrieval (Prometheus)

Navigate to <http://localhost:9090>.

Enter the query: `cpu_usage_percentage`

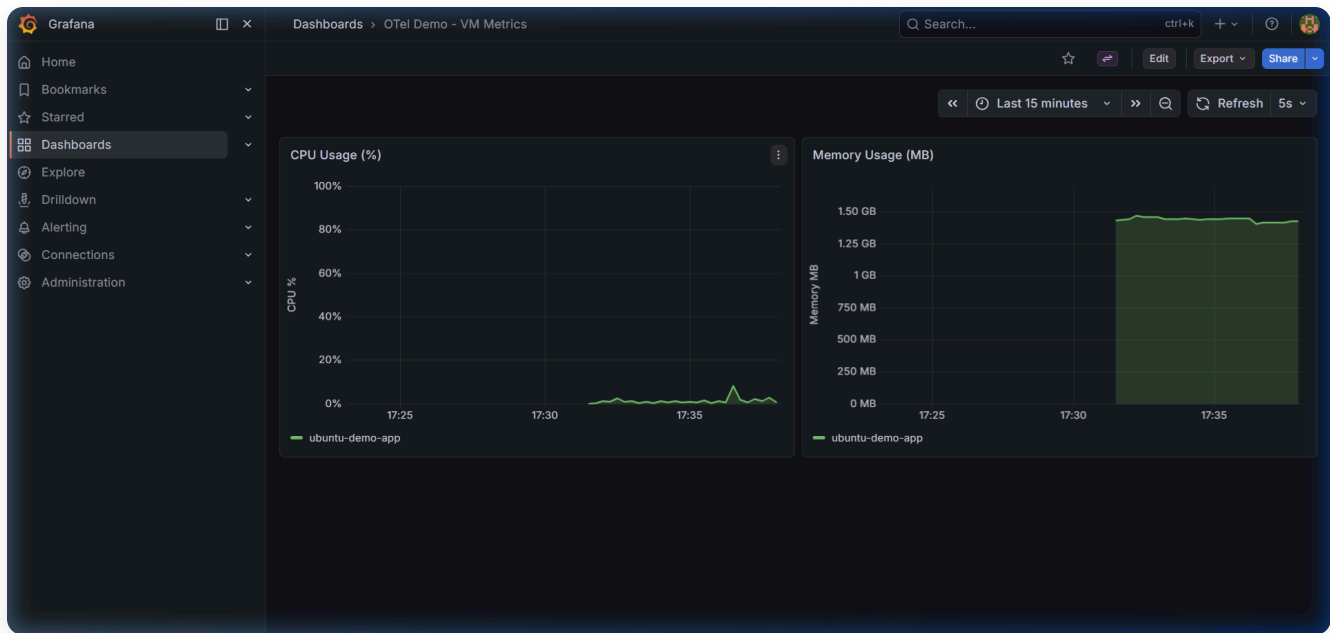


(Visual verification screenshot verified in demo)

Step 4: Final Visual Dashboard (Grafana)

Navigate to <http://localhost:3000> (Login: **admin** / **admin**).

Open **Dashboards** -> **OTel Demo - VM Metrics**.



(Visual dashboard verified in demo)



4. Team FAQ & Troubleshooting Lessons

During development, we solved these common real-world OTEL issues:

1. **The "Unit Suffix" Trap:** The OTEL-Prometheus exporter appends `/v1/metrics` automatically to metric names if a unit (like `%`) is present. We removed units from the code to keep query names clean (`cpu_usage_percentage`).
2. **Core vs. Contrib:** The base OTEL image is very small and lacks some exporters. We used the **Contrib** image to get Prometheus support.
3. **gRPC Protocol:** Python's newer OTEL SDK requires `insecure=True` and a specifically formatted endpoint (`http://otel-collector:4317`) to resolve DNS correctly in Docker.



5. Next Steps: Scaling to AWS & Azure

To move this to production:

- Replace the local `prometheus` container with **Amazon Managed Prometheus (AMP)** or **Azure Monitor**.
- Simply update the `exporters` in `otel-collector-config.yaml`.
- **Profit:** Your application code stays exactly the same as in this demo!